

HTTP Client Setup in Angular

Connecting your Angular app to the outside world using HTTP.

1. Why HTTP Client?

Every real-world web application needs to talk to a server. Your Angular app may need to load a list of products from a database, submit a login form, or delete a record. All of this communication happens over the **HTTP protocol** — the same protocol your browser uses to load web pages.

Angular provides a built-in tool called `HttpClient` that handles this communication cleanly and consistently. Without it, you would have to write raw `fetch()` or `XMLHttpRequest` calls manually, which is tedious and error-prone.

Analogy — The Restaurant

Think of your Angular app as a restaurant:

- The **Component** is the dining area — it shows things to the user.
- The **Service** is the waiter — it takes requests and brings back results.
- **HttpClient** is the delivery system — it goes outside the restaurant to fetch ingredients (data).
- The **API server** is the external supplier — it stores and provides the data.

A customer (user) asks for a dish → the waiter (service) tells the delivery system (HttpClient) → the delivery system goes to the supplier (API) → brings back the data → the waiter hands it to the dining area (component) → the customer sees the result.

What is a REST API?

A **REST API** (Representational State Transfer) is a standard way for two systems to communicate. Your Angular app sends an HTTP request to a URL (called an *endpoint*), and the server responds with data — usually in **JSON** format.

HTTP defines several **methods** (also called verbs) to describe what you want to do with the data:

Method	Purpose	Real-world Example
GET	Retrieve / read data	Load a list of users
POST	Create new data	Register a new user
PUT	Replace an entire record	Update all fields of a user profile
PATCH	Update specific fields only	Change only the user's email
DELETE	Remove a record	Delete a user account

What is JSONPlaceholder?

Throughout this module we use <https://jsonplaceholder.typicode.com> — a free, public fake REST API designed for testing. It gives you realistic responses without needing your own backend server.

The most common endpoints are:

- `/users` — A list of users
- `/posts` — A list of blog posts
- `/todos` — A list of to-do items

2. Setting Up HttpClient

Before using `HttpClient` in any component or service, you must register it once with Angular's dependency injection system. The approach differs slightly depending on whether your project is a **standalone app** (Angular 17+) or a **module-based app** (Angular 15 and below).

Standalone vs Module-Based — Which do I have?

Open `src/main.ts`. If you see `bootstrapApplication(AppComponent, appConfig)`, your app is **standalone**. If you see `bootstrapModule(AppModule)`, it is **module-based**. Both approaches are shown below.

Option A — Standalone App (Modern Angular)

APP.CONFIG.TS

```
import { ApplicationConfig } from '@angular/core';
import { provideHttpClient } from '@angular/common/http'; // ← import this

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient() // ← register it here – once for the whole app
  ]
};
```

Option B — Module-Based App

APP.MODULE.TS

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http'; // ← import this
import { AppComponent } from './app.component';

@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    HttpClientModule // ← add it to imports array
  ]
})
```

```
],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

Good Practice — Use a Service for HTTP calls

Angular encourages keeping HTTP logic inside a **service**, not directly inside a component. This way, if three different components need the same data, they all call the same service — you do not duplicate the HTTP code. Components only deal with *displaying* data.

3. Fetching Data — GET Request

The `GET` method retrieves data from a server without modifying anything. This is the most frequently used HTTP method. We'll build this up in steps.

Step 1 — Create a Service

First, define the shape of the data using a TypeScript **interface**, then create a service that uses `HttpClient` to fetch it.

USER.SERVICE.TS

```
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable } from 'rxjs';  
  
// Define the shape of a User object  
export interface User {  
  id: number;  
  name: string;  
  email: string;  
}  
  
@Injectable({ providedIn: 'root' })  
export class UserService {  
  
  private apiUrl = 'https://jsonplaceholder.typicode.com/users';
```

```
constructor(private http: HttpClient) { }

// Returns an Observable that will emit an array of Users
getUsers(): Observable<User[]> {
  return this.http.get<User[]>(this.apiUrl);
}

}
```

What is an Observable?

An `Observable` (from the RxJS library) is like a *promise to deliver data in the future*. When you call `this.http.get()`, Angular does **not** send the request immediately. It just prepares it. The request is only sent when you call `.subscribe()` on the Observable — that is your way of saying "yes, please go and fetch it now."

Observables are covered in detail in Topic 12.2. For now, just remember: **always call `.subscribe()` to trigger the request.**

Step 2 — Use the Service in a Component

USER-LIST.COMPONENT.TS

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { UserService, User } from './user.service';

@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [CommonModule],
  template: `
    <h2>User List</h2>
    <p *ngFor="let user of users">{{ user.name }} – {{ user.email }}</p>
  `
})
export class UserListComponent implements OnInit {

  users: User[] = []; // will hold the fetched list
```

```
constructor(private userService: UserService) { }

ngOnInit() {
  // .subscribe() triggers the actual HTTP request
  this.userService.getUsers().subscribe({
    next: (data) => {
      this.users = data; // store the result
    },
    error: (err) => {
      console.error('Request failed:', err);
    }
  });
}
```

Output / Result:

User List

Leanne Graham – sincere@april.biz
Ervin Howell – shanna@melissa.tv
Clementine Bauch – nathan@yesenia.net
... 7 more users

Step 3 — Adding Loading and Error States

In production apps, data does not appear instantly — there is a brief network delay. It is important to show a loading indicator while the request is in progress, and a clear error message if it fails. Let's improve the component.

USER-LIST.COMPONENT.TS — IMPROVED

```
@Component({
  selector: 'app-user-list',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div *ngIf="loading">Loading users...</div>
    <div *ngIf="errorMessage" style="color: red;">{{ errorMessage }}</div>
    <div *ngIf="!loading && !errorMessage">
```

```
        <p *ngFor="let user of users">{{ user.name }} – {{ user.email }}</p>
    </div>
    ,
  })
  export class UserListComponent implements OnInit {

    users: User[] = [];
    loading = false;
    errorMessage = '';

    constructor(private userService: UserService) { }

    ngOnInit() {
      this.loading = true; // show spinner

      this.userService getUsers().subscribe({
        next: (data) => {
          this.users = data;
          this.loading = false; // hide spinner
        },
        error: (err) => {
          this.errorMessage = 'Failed to load users. Please try again.';
          this.loading = false;
        }
      });
    }
  }
}
```

Output / Result:

While request is in progress:

Loading users...

After success:

Leanne Graham – sincere@april.biz

Ervin Howell – shanna@melissa.tv

...

If request fails (e.g., no internet):

Failed to load users. Please try again.

Step 4 — Fetching a Single Item by ID

To fetch one specific record, append its ID to the URL. The URL pattern `/users/1` returns only the user whose `id` is 1.

USER.SERVICE.TS — ADD THIS METHOD

```
// Add to UserService
getUserById(id: number): Observable<User> {
  return this.http.get<User>(`${this.apiUrl}/${id}`);
  //           ↑ template literal builds: /users/1
}
```

USER-DETAIL.COMPONENT.TS

```
export class UserDetailComponent implements OnInit {
  user: User | null = null;

  constructor(private userService: UserService) { }

  ngOnInit() {
    this.userService.getUserById(1).subscribe({
      next: (data) => {
        this.user = data;
      }
    });
  }
}
```

Output / Result:

URL called: GET <https://jsonplaceholder.typicode.com/users/1>

Response (JSON):

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "sincere@april.biz"
}
```

4. Creating Data — POST Request

The `POST` method sends data to the server to create a new record. Unlike `GET`, a POST request includes a **request body** — the data you want to save.

Step 1 — Add a `createUser` Method to the Service

USER.SERVICE.TS — ADD THIS METHOD

```
// Omit<User, 'id'> means: a User object WITHOUT the 'id' field
// (the server assigns the ID automatically)
createUser(user: Omit<User, 'id'>): Observable<User> {
  return this.http.post<User>(this.apiUrl, user);
  //                                URL ↑      body ↑
}
```

Step 2 — Use It in a Component

CREATE-USER.COMPONENT.TS

```
@Component({
  selector: 'app-create-user',
  standalone: true,
  imports: [CommonModule, FormsModule],
  template: `
    <h3>Add New User</h3>
    <input [(ngModel)]="newUser.name" placeholder="Full Name" />
    <input [(ngModel)]="newUser.email" placeholder="Email" />
    <button (click)="createUser()">Create</button>
    <p *ngIf="successMessage">{{ successMessage }}</p>
  `
})
export class CreateUserComponent {

  newUser = { name: '', email: '' };
  successMessage = '';

  constructor(private userService: UserService) { }

  createUser() {
    this.userService.createUser(this.newUser).subscribe({
      next: (response) => {
```

```

    this.successMessage = 'User created! Server assigned ID: ' + response.id;
    this.newUser = { name: '', email: '' }; // clear the form
  },
  error: (err) => {
    console.error('Failed to create user:', err);
  }
});
}
}

```

Output / Result:

Request sent:

POST https://jsonplaceholder.typicode.com/users
 Body: { "name": "John Doe", "email": "john@example.com" }

Response from server:

{ "id": 11, "name": "John Doe", "email": "john@example.com" }

Why does the server return the object with an ID?

When you create a record, the server stores it in the database and assigns a unique ID. The server then sends back the full saved object (including the new ID). This is the standard REST convention. In a real app, you would use this returned ID for future updates or deletes.

5. Updating Data — PUT and PATCH

There are two methods to update an existing record. Choosing the right one matters.

Method	What you send	What the server does	Use when...
PUT	The complete object (all fields)	Replaces the entire record	You want to update everything at once
PATCH			

Method	What you send	What the server does	Use when...
	Only the fields you want to change	Merges changes, keeps the rest	You want to change just one or two fields

Analogy — Editing a Form

Imagine you have a printed form with ten fields. **PUT** is like filling out a brand new form and handing it in — all ten fields. **PATCH** is like using white-out on just the "email" field and writing the new value — everything else stays the same.

PUT — Full Replacement

USER.SERVICE.TS — ADD THIS METHOD

```
updateUser(id: number, user: User): Observable<User> {
  return this.http.put<User>(`${this.apiUrl}/${id}`, user);
}
```

USAGE IN A COMPONENT

```
const updatedUser: User = { id: 1, name: 'Jane Doe', email: 'jane@example.com' };

this.userService.updateUser(1, updatedUser).subscribe({
  next: (response) => {
    console.log('Updated:', response);
  }
});
```

Output / Result:

Request: PUT https://jsonplaceholder.typicode.com/users/1

Body: { "id": 1, "name": "Jane Doe", "email": "jane@example.com" }

Response: { "id": 1, "name": "Jane Doe", "email": "jane@example.com" }

PATCH — Partial Update

USER.SERVICE.TS — ADD THIS METHOD

```
// Partial<User> means: any subset of User fields is acceptable
patchUser(id: number, changes: Partial<User>): Observable<User> {
  return this.http.patch<User>(`${this.apiUrl}/${id}`, changes);
}
```

USAGE IN A COMPONENT

```
// Only update the email – name and other fields are untouched
this.userService.patchUser(1, { email: 'newemail@example.com' }).subscribe({
  next: (response) => {
    console.log('Email updated:', response.email);
  }
});
```

Output / Result:

Request: PATCH https://jsonplaceholder.typicode.com/users/1

Body: { "email": "newemail@example.com" }

Response: { "id": 1, "name": "Leanne Graham", "email": "newemail@example.com" }
→ name was NOT changed, only email was updated

6. Deleting Data — DELETE Request

The `DELETE` method removes a record from the server. You identify which record to delete by including its ID in the URL.

USER.SERVICE.TS — ADD THIS METHOD

```
deleteUser(id: number): Observable<any> {  
  return this.http.delete(`${this.apiUrl}/${id}`);  
  // No body is needed for DELETE – the ID in the URL is enough  
}
```

USAGE IN A COMPONENT

```
export class UserListComponent {  
  users: User[] = [];  
  
  constructor(private userService: UserService) { }  
  
  deleteUser(userId: number) {  
    if (!confirm('Are you sure you want to delete this user?')) return;  
  
    this.userService.deleteUser(userId).subscribe({  
      next: () => {  
        // Remove from the local array so the UI updates instantly  
        this.users = this.users.filter(u => u.id !== userId);  
        console.log('User deleted successfully');  
      },  
      error: (err) => {  
        console.error('Delete failed:', err);  
      }  
    });  
  }  
}
```

Output / Result:

Request: DELETE https://jsonplaceholder.typicode.com/users/1

Response: 200 OK (empty body – the record no longer exists)

Why filter the local array?

After a successful DELETE, the server has removed the record, but your `users` array in memory still contains it. You must also remove it from the local array so the template re-renders and the user disappears from the screen. This is the standard approach instead of calling `getUsers()` again (which would make an extra network request).

7. Sending Query Parameters

Query parameters let you filter, search, or paginate data without changing the base URL. They appear after a `?` in the URL, for example: `/users?name=John&limit=10`.

Angular provides the `HttpParams` class to build these parameters in a clean, chainable way.

USER.SERVICE.TS — SEARCH EXAMPLE

```
import { HttpParams } from '@angular/common/http';

searchUsers(searchTerm: string): Observable<User[]> {
  const params = new HttpParams().set('name', searchTerm);
  //                               ↑ builds ?name=searchTerm

  return this.http.get<User[]>(this.apiUrl, { params });
  //                               ↑ pass as options object
}
```

Output / Result:

Generated URL: `https://jsonplaceholder.typicode.com/users?name=John`

For multiple parameters, chain the `.set()` calls:

MULTIPLE PARAMETERS

```
searchProducts(category: string, minPrice: number): Observable<any[]> {
  const params = new HttpParams()
    .set('category', category)
    .set('minPrice', minPrice.toString()); // params must be strings
}
```

```
return this.http.get<any[]>(this.apiUrl, { params });
}
```

Output / Result:

Generated URL: /products?category=electronics&minPrice=100

8. Complete Example — User Manager

The following is a complete, working example that brings together the service with all five HTTP methods and uses a single component to display and manage users. This is the structure you would use in a real project.

The Service

USER.SERVICE.TS

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface User {
  id: number;
  name: string;
  email: string;
}

@Injectable({ providedIn: 'root' })
export class UserService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/users';

  constructor(private http: HttpClient) { }

  // GET – all users
  getUsers(): Observable<User[]> {
    return this.http.get<User[]>(this.apiUrl);
  }
}
```

```
// GET – single user
getUserById(id: number): Observable<User> {
  return this.http.get<User>(`${this.apiUrl}/${id}`);
}

// POST – create
createUser(user: Omit<User, 'id'>): Observable<User> {
  return this.http.post<User>(this.apiUrl, user);
}

// PUT – full update
updateUser(id: number, user: User): Observable<User> {
  return this.http.put<User>(`${this.apiUrl}/${id}`, user);
}

// PATCH – partial update
patchUser(id: number, changes: Partial<User>): Observable<User> {
  return this.http.patch<User>(`${this.apiUrl}/${id}`, changes);
}

// DELETE
deleteUser(id: number): Observable<any> {
  return this.http.delete(`${this.apiUrl}/${id}`);
}

// GET with query params
searchUsers(name: string): Observable<User[]> {
  const params = new HttpParams().set('name', name);
  return this.http.get<User[]>(this.apiUrl, { params });
}
}
```

The Component

USER-MANAGER.COMPONENT.TS

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { UserService, User } from '../user.service';

@Component({
  selector: 'app-user-manager',
  standalone: true,
  imports: [CommonModule, FormsModule],
```

```

template: `
  <h2>User Manager</h2>

  <!-- Loading / error states -->
  <p *ngIf="loading">Loading...</p>
  <p *ngIf="message" [style.color]="messageColor">{{ message }}</p>

  <!-- Create form -->
  <div>
    <input [(ngModel)]="newUser.name" placeholder="Name" />
    <input [(ngModel)]="newUser.email" placeholder="Email" />
    <button (click)="createUser()">Add User</button>
  </div>

  <!-- User list -->
  <ul>
    <li *ngFor="let user of users">
      {{ user.id }}. {{ user.name }} - {{ user.email }}
      <button (click)="deleteUser(user.id)">Delete</button>
    </li>
  </ul>
`
})
export class UserManagerComponent implements OnInit {

  users: User[] = [];
  newUser = { name: '', email: '' };
  loading = false;
  message = '';
  messageColor = 'green';

  constructor(private userService: UserService) { }

  ngOnInit() {
    this.loading = true;
    this.userService.getUsers().subscribe({
      next: (data) => {
        this.users = data.slice(0, 5); // show first 5 for brevity
        this.loading = false;
      },
      error: () => {
        this.setMessage('Failed to load users.', 'red');
        this.loading = false;
      }
    });
  }
}

```

```
createUser() {
  if (!this.newUser.name || !this.newUser.email) return;

  this.userService.createUser(this.newUser).subscribe({
    next: (created) => {
      this.users.push(created);
      this.newUser = { name: '', email: '' };
      this.setMessage('User created successfully!', 'green');
    }
  });
}

deleteUser(id: number) {
  this.userService.deleteUser(id).subscribe({
    next: () => {
      this.users = this.users.filter(u => u.id !== id);
      this.setMessage('User deleted.', 'green');
    }
  });
}

private setMessage(text: string, color: string) {
  this.message = text;
  this.messageColor = color;
  setTimeout(() => this.message = '', 3000); // auto-hide after 3s
}
}
```

Output / Result:

User Manager

[Name input] [Email input] [Add User button]

- 1. Leanne Graham – sincere@april.biz [Delete]
- 2. Ervin Howell – shanna@melissa.tv [Delete]
- 3. Clementine Bauch – nathan@yesenia.net [Delete]
- 4. Patricia Lebsack – julianne.okeefe@kory.org [Delete]
- 5. Chelsey Dietrich – lucio_hettinger@annie.ca [Delete]

Interview Questions

Q1. What is HttpClient in Angular, and why do we use it?

`HttpClient` is Angular's built-in service for making HTTP requests to backend APIs. It abstracts the complexity of raw browser APIs (`XMLHttpRequest` / `fetch`) and provides features such as automatic JSON parsing, TypeScript type safety via generics, Observable-based responses, and built-in support for interceptors. We use it because it integrates naturally with Angular's dependency injection and the RxJS reactive programming model.

Q2. What is the difference between GET and POST?

`GET` retrieves data from the server and does not modify any resource. It has no request body. `POST` sends data to the server to create a new resource and includes a **request body** containing the data. `GET` requests can be cached and bookmarked; `POST` requests cannot.

Q3. What is the difference between PUT and PATCH?

`PUT` requires you to send the **complete** object — the server replaces the existing record entirely. `PATCH` allows you to send only the **fields you want to change** — the server merges them into the existing record. Use `PUT` when updating all fields, and `PATCH` when updating one or two specific fields.

Q4. Why do we need to call `.subscribe()` on HTTP requests?

`HttpClient` methods return **cold Observables** — they do nothing until someone subscribes. Calling `.subscribe()` is what actually triggers the HTTP network request. Without it, the request is never sent. This is different from Promises, which execute immediately on creation.

Q5. Where should HTTP logic live — in a component or a service? Why?

Always in a **service**. Components are responsible for the UI — they should only display data and respond to user interactions. Services handle data fetching, business logic, and API communication. This separation ensures the code is *reusable* (multiple components can use the same service), *testable* (the service can be unit-tested independently), and *maintainable* (changes to the API URL or request logic are made in one place).

Q6. How do you pass query parameters in an Angular HTTP request?

Use the `HttpParams` class from `@angular/common/http` :

```
const params = new HttpParams()
  .set('category', 'books')
  .set('limit', '10');

this.http.get(url, { params });
```

This generates: `/api/items?category=books&limit=10`. Never concatenate query strings manually — `HttpParams` handles encoding automatically.

Q7. What does `provideHttpClient()` do in a standalone Angular app?

It registers `HttpClient` and its required dependencies with Angular's root injector. This replaces the older `HttpClientModule` import approach. It must be called once in `app.config.ts`, after which `HttpClient` is available for injection anywhere in the application.

Practical Exercise

Assignment: Posts Manager

Build a simple blog post manager using the JSONPlaceholder API endpoint `https://jsonplaceholder.typicode.com/posts`. A post has the shape: `{ id, title, body, userId }`.

Requirements:

1. Create `post.service.ts` with the following methods:
 - `getPosts()` — Fetch all posts (display only the first 10)
 - `getPostById(id)` — Fetch a single post by ID
 - `createPost(post)` — Create a new post via POST
 - `updatePost(id, post)` — Full update via PUT
 - `deletePost(id)` — Delete a post via DELETE
2. Create `post-list.component.ts` that:
 - Displays the list of posts (title only) on load
 - Shows a loading indicator while fetching

- Has a "Delete" button next to each post that removes it from the list
3. Create `create-post.component.ts` with a form (title + body inputs) and an "Add Post" button that calls `createPost()` and adds the result to the displayed list.
 4. **Bonus:** Add a search input that uses `HttpParams` to filter posts by `userId`.

Tip

Use `https://jsonplaceholder.typicode.com` — it accepts all request types and returns realistic responses. Note that it simulates responses only; no data is actually stored on the server.

